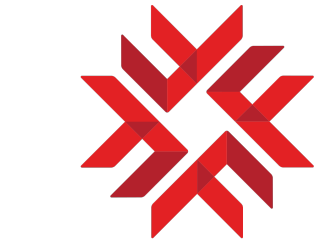
Tensorflow II

Object Detection

Fatemeh Dehrouyeh



Contents

1. Object Detection
   - Motivation
   - Bounding Boxes
   - Intersection over Union
   - Non-Maximum Suppression
   - Multiple Classes
   - Detector Architectures

Current Section

1 Object Detection
- Motivation
- Bounding Boxes
- Intersection over Union
- Non-Maximum Suppression
- Multiple Classes
- Detector Architectures

Current Subsection

1. Object Detection
   - Motivation
   - Bounding Boxes
   - Intersection over Union
   - Non-Maximum Suppression
   - Multiple Classes
   - Detector Architectures

From Classification to Object Detection

Image classification answers:

What is in the image?

Object detection answers two questions:

What objects are in the image?

Where are they located?

Key Idea

Object detection combines **classification** and **localization**.

Classification vs. Detection

Task	Output	Example
Classification	One label	cat
Localization	One box + label	cat at location
Detection	Multiple boxes + labels	cat, dog, car

Detection Output

An object detector predicts object classes, bounding boxes, and confidence scores.

What Does an Object Detector Predict?

For each detected object, the model predicts:

- **class label**: what object it is,
- **bounding box**: where the object is,
- **confidence score**: how confident the model is.

A detection result may look like:

$$(\text{cat},\ 0.94,\ x_{\min}, y_{\min}, x_{\max}, y_{\max})$$

Important

Unlike classification, object detection can return multiple predictions for one image.

Current Subsection

1. Object Detection
   - Motivation
   - Bounding Boxes
   - Intersection over Union
   - Non-Maximum Suppression
   - Multiple Classes
   - Detector Architectures

What Is a Bounding Box?

A **bounding box** is a rectangle that surrounds an object in an image.

It is commonly represented using four numbers:

$$(x_{\text{min}}, y_{\text{min}}, x_{\text{max}}, y_{\text{max}})$$

where:

- $(x_{\text{min}}, y_{\text{min}})$ is the top-left corner,
- $(x_{\text{max}}, y_{\text{max}})$ is the bottom-right corner.

Key Idea

A bounding box tells the model where the object is located.

TensorFlow Bounding Box Convention

We often write a bounding box as

$$(x_{\text{min}}, y_{\text{min}}, x_{\text{max}}, y_{\text{max}})$$

However, many TensorFlow image functions use the order

$$\boxed{[y_{\text{min}}, x_{\text{min}}, y_{\text{max}}, x_{\text{max}}]}$$

and often expect the coordinates to be normalized to

$$[0, 1].$$

Important

The same bounding box may therefore have a different coordinate order depending on the library or function being used.

Image Example

```python
import requests
import tensorflow as tf
import matplotlib.pyplot as plt

url = "https://static.wixstatic.com/media/4b3c31_4ceabbcd4a9c4048b0383cbbb435bac3~mv2.jpg/

response = requests.get(url)

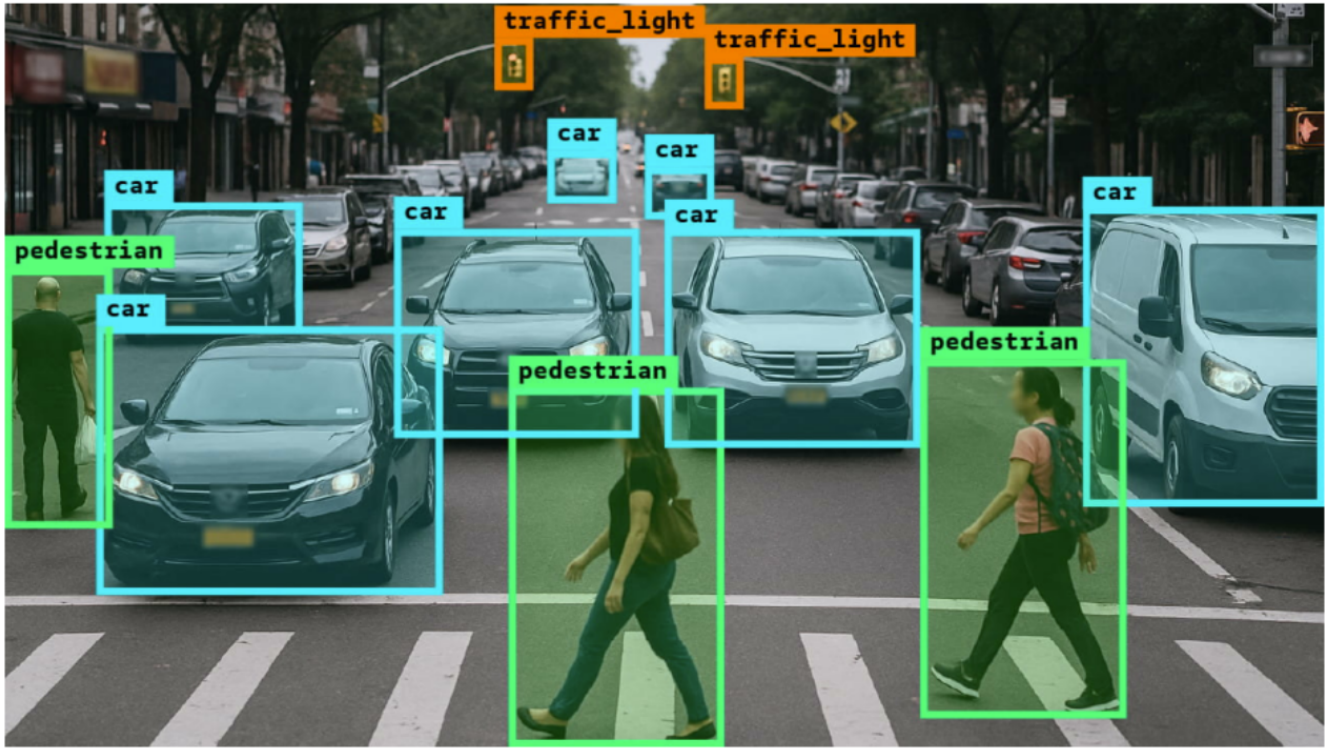
# Decode directly as a TensorFlow tensor
image_tf = tf.io.decode_image(
    response.content,
    channels=3,
    expand_animations=False
)

print("Type:", type(image_tf))
print("Shape:", image_tf.shape)
print("Dtype:", image_tf.dtype)

plt.figure(figsize=(12, 7))
plt.imshow(image_tf)
plt.axis("off")
plt.show()
```

Image Example—Result

```
Type: <class 'tensorflow.python.framework.ops.EagerTensor'>
Shape: (720, 1280, 3)
Dtype: <dtype: 'uint8'>
```



Code

```
# Get image dimensions using TensorFlow

image_shape = tf.shape(image_tf)

image_height = image_shape[0]
image_width = image_shape[1]
channels = image_shape[2]

print("Height:", image_height.numpy())
print("Width:", image_width.numpy())
print("Channels:", channels.numpy())
```

```
Height: 720
Width: 1280
Channels: 3
```

TensorFlow Bounding Box Example

For the selected car, we first define the box in pixel coordinates:

$$(x_{\text{min}}, y_{\text{min}}, x_{\text{max}}, y_{\text{max}})$$

Then we:

1. convert the coordinates to normalized values,
2. reorder them into TensorFlow format,
3. draw the box using `tf.image.draw_bounding_boxes()`.

TensorFlow Format

$$[y_{\text{min}}, x_{\text{min}}, y_{\text{max}}, x_{\text{max}}]$$

Bounding Box Formats

Bounding boxes can be represented in several ways.

$$(x_{\text{min}}, y_{\text{min}}, x_{\text{max}}, y_{\text{max}})$$

uses the two opposite corners.

$$(x_{\text{min}}, y_{\text{min}}, w, h)$$

uses the top-left corner together with width and height.

$$(x_{\text{center}}, y_{\text{center}}, w, h)$$

uses the box center together with width and height.

Important

Always check the expected bounding-box format and coordinate order for the dataset, model, or TensorFlow function being used.

Code

```
# Define the bounding box

xmin = tf.constant(95.0)
ymin = tf.constant(315.0)
xmax = tf.constant(430.0)
ymax = tf.constant(570.0)
```

This is the common $[x_{\text{min}}, y_{\text{min}}, x_{\text{max}}, y_{\text{max}}]$ format.

Code

```
# Convert xyxy to width and height

box_width = xmax - xmin
box_height = ymax - ymin

print("Box width:", box_width.numpy())
print("Box height:", box_height.numpy())
```

```
Box width: 335.0
Box height: 255.0
```

So: $w = x_{\max} - x_{\min}, \quad h = y_{\max} - y_{\min}$

Code

```
# Convert to xywh

box_xywh = tf.stack([
    xmin,
    ymin,
    box_width,
    box_height
])

print("xywh:")
print(box_xywh.numpy())
```

```
xywh:
[ 95. 315. 335. 255.]
```

This represents: $[x_{\text{min}}, y_{\text{min}}, w, h]$

Code

Some detectors use: $[x_c, y_c, w, h]$

```
# Convert to center format

x_center = xmin + box_width / 2
y_center = ymin + box_height / 2

box_center = tf.stack([
    x_center,
    y_center,
    box_width,
    box_height
])

print("Center format:")
print(box_center.numpy())
```

```
Center format:
[262.5 442.5 335.  255. ]
```

Normalized Bounding Boxes

Some models and TensorFlow functions use normalized bounding-box coordinates. Instead of pixel coordinates, values are scaled to

$$0.0 \leq x, y \leq 1.0$$

For an image with width W and height H:

$$x_{\text{norm}} = \frac{x}{W} \qquad y_{\text{norm}} = \frac{y}{H}$$

Why Normalize?

Normalized coordinates make the box representation independent of the original image dimensions.

Code

```
image_width_f = tf.cast(image_width, tf.float32)
image_height_f = tf.cast(image_height, tf.float32)

xmin_norm = xmin / image_width_f
xmax_norm = xmax / image_width_f

ymin_norm = ymin / image_height_f
ymax_norm = ymax / image_height_f

print("Normalized xmin:", xmin_norm.numpy())
print("Normalized ymin:", ymin_norm.numpy())
print("Normalized xmax:", xmax_norm.numpy())
print("Normalized ymax:", ymax_norm.numpy())
```

```
Normalized xmin: 0.07421875
Normalized ymin: 0.4375
Normalized xmax: 0.3359375
Normalized ymax: 0.7916667
```

Casting prevents type conflicts and makes decimal calculations explicit

Code

Many TensorFlow image functions expect: $[y_{\text{min}}, x_{\text{min}}, y_{\text{max}}, x_{\text{max}}]$. So construct it as:

```
box_tf = tf.stack([
    ymin_norm,
    xmin_norm,
    ymax_norm,
    xmax_norm
])

print("TensorFlow box:")
print(box_tf.numpy())
```

```
TensorFlow box:
[0.4375     0.07421875 0.7916667  0.3359375 ]
```

Draw the box using TensorFlow

`tf.image.draw_bounding_boxes()` expects: (batch, number_of_boxes, 4). So first let's add those dimensions:

```
boxes = tf.reshape(
    box_tf,
    shape=(1, 1, 4)
)

print("Boxes shape:", boxes.shape)
```

```
Boxes shape: (1, 1, 4)
```

(1, 1, 4) : 1 image in the batch, 1 bounding box for that image, and 4 coordinates describing the box.

All images in the same batch must have the same number of box entries in the tensor. Different batches can have different numbers of bounding boxes. When training an object detection model, we must be careful when creating batches because different images can contain different numbers of objects.

Draw the box using TensorFlow—Continued

Now prepare the image:

```
image_float = tf.image.convert_image_dtype(
    image_tf,
    tf.float32
)

image_batch = tf.expand_dims(
    image_float,
    axis=0
)

print("Image batch shape:", image_batch.shape)
```

```
Image batch shape: (1, 720, 1280, 3)
```

`tf.expand_dims(axis=0)` adds a batch dimension: $(720, 1280, 3) \rightarrow (1, 720, 1280, 3)$, meaning one image in the batch.

Draw the box using TensorFlow—Continued

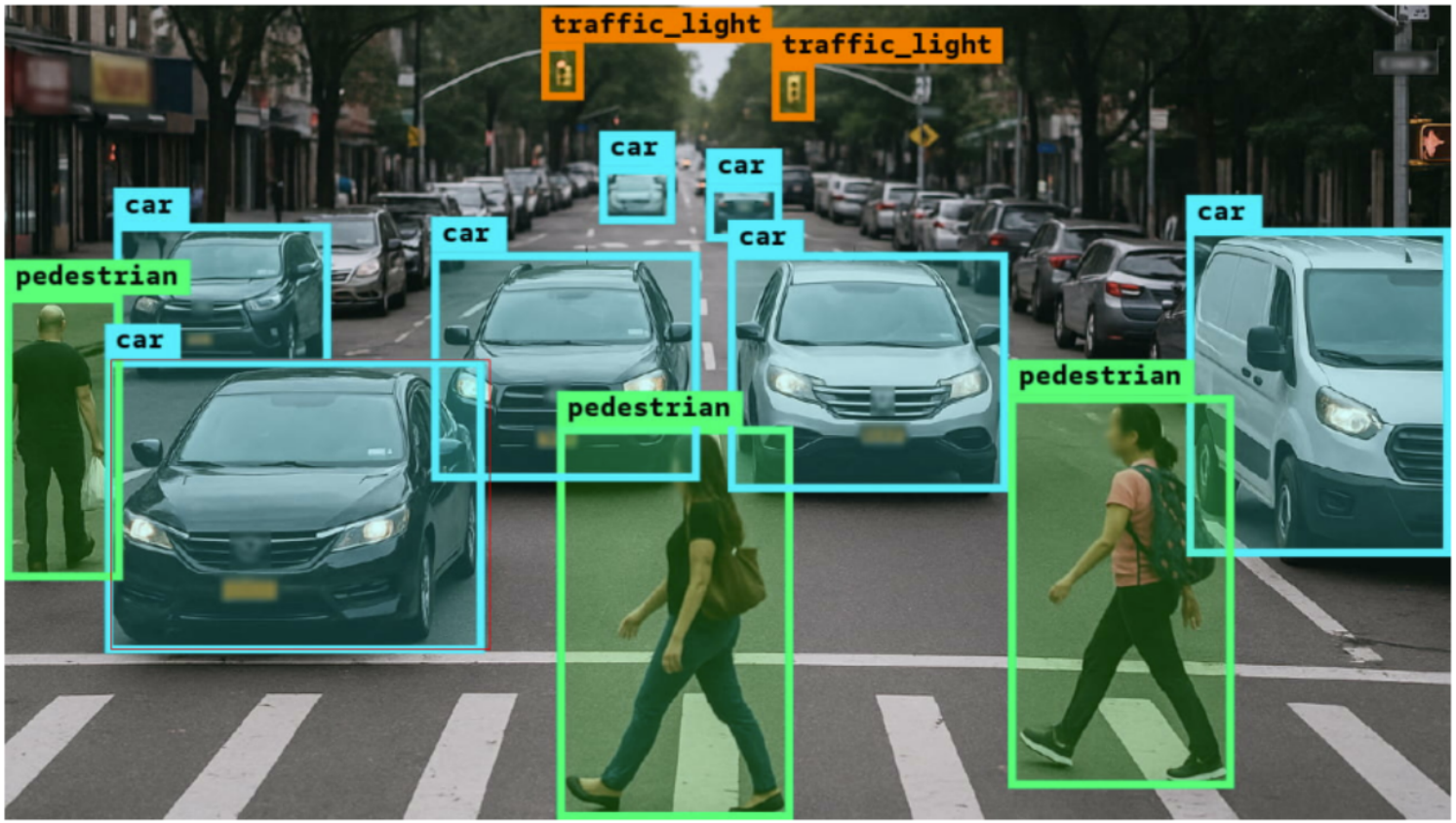
```
image_with_box = tf.image.draw_bounding_boxes(
    image_batch,
    boxes,
    colors=[[1.0, 0.0, 0.0]]
)

plt.figure(figsize=(12, 7))
plt.imshow(image_with_box[0])
plt.axis("off")
plt.show()
```

tf.image.draw_bounding_boxes() requires bounding-box coordinates normalized between 0 and 1; without normalization, pixel coordinates would be interpreted incorrectly, causing the box to appear in the wrong location or outside the image.

Result

...


traffic_light
traffic_light
car
car
car
car
car
car
pedestrian
car
pedestrian
pedestrian

Another Approach

```
import matplotlib.pyplot as plt
import matplotlib.patches as patches

fig, ax = plt.subplots(figsize=(10, 6))

ax.imshow(image_tf)

rect = patches.Rectangle(
    (xmin, ymin),
    box_width,
    box_height,
    fill=False,
    linewidth=3
)

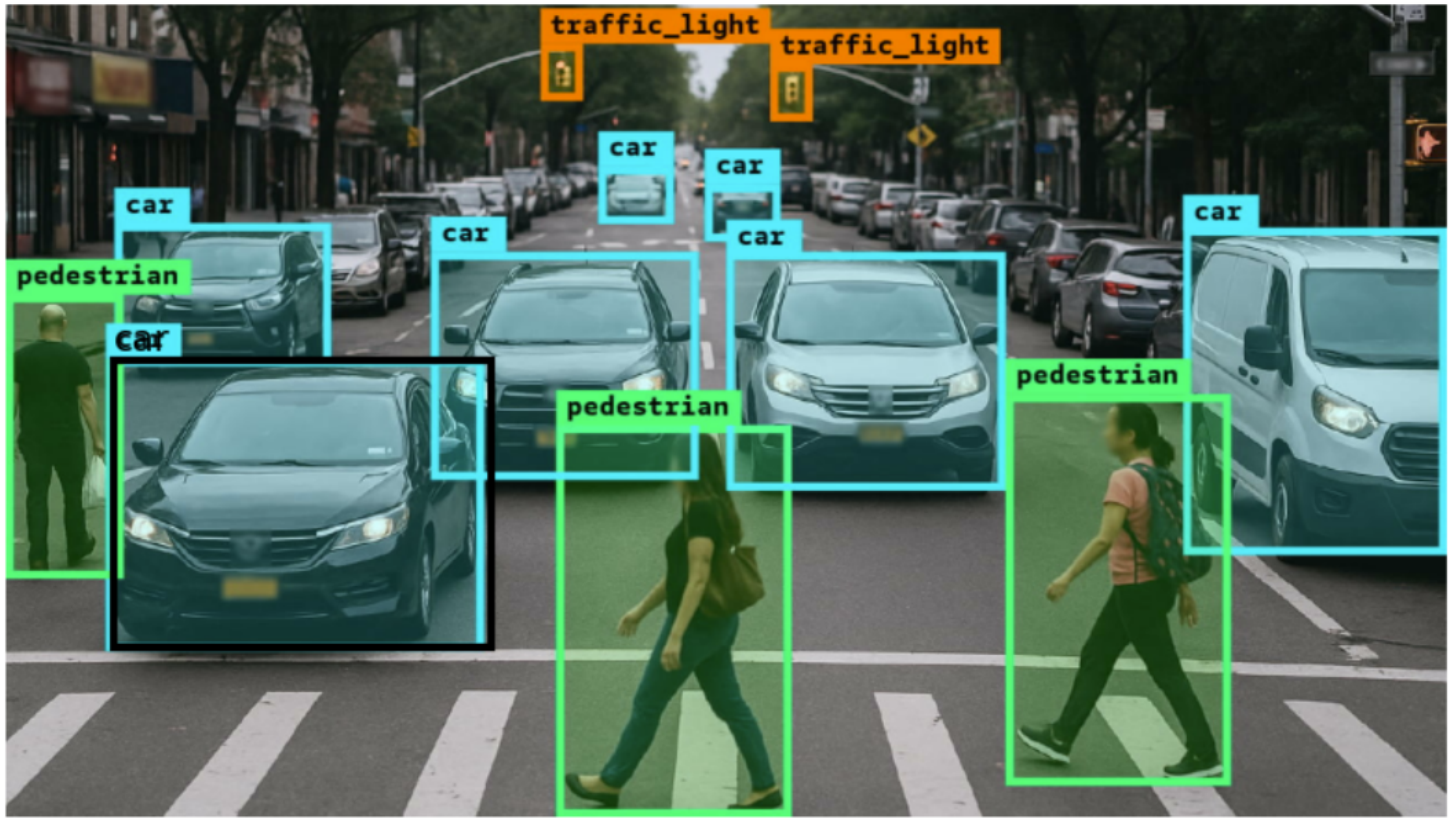
ax.add_patch(rect)

ax.text(
    xmin,
    ymin - 10,
    "car",
    fontsize=14
)

ax.axis("off")
plt.show()
```

Result

...


traffic_light
traffic_light
car
car
car
car
car
car
pedestrian
pedestrian
pedestrian

Current Subsection

1. Object Detection
   - Motivation
   - Bounding Boxes
   - Intersection over Union
   - Non-Maximum Suppression
   - Multiple Classes
   - Detector Architectures

Intersection over Union (IoU)

Object detection needs a way to measure how well a predicted bounding box matches the true bounding box.

The most common measure is **Intersection over Union (IoU)**:

$$\text{IoU} = \frac{\text{Area of Intersection}}{\text{Area of Union}}$$

- $\text{IoU} = 0$: no overlap
- $\text{IoU} = 1$: perfect overlap
- larger IoU means a better localization

Example

An IoU threshold such as 0.5 is often used to decide whether a predicted box sufficiently matches a ground-truth box.

Code

```python
import tensorflow as tf

# TensorFlow box format:
# [ymin, xmin, ymax, xmax]

box1 = tf.constant([0.20, 0.20, 0.70, 0.70], dtype=tf.float32)
box2 = tf.constant([0.40, 0.40, 0.90, 0.90], dtype=tf.float32)

def compute_iou(box1, box2):

    # Intersection coordinates
    ymin = tf.maximum(box1[0], box2[0])
    xmin = tf.maximum(box1[1], box2[1])
    ymax = tf.minimum(box1[2], box2[2])
    xmax = tf.minimum(box1[3], box2[3])

    # Intersection size
    intersection_height = tf.maximum(0.0, ymax - ymin)
    intersection_width = tf.maximum(0.0, xmax - xmin)

    intersection_area = intersection_height * intersection_width

    # Box areas
    box1_area = (box1[2] - box1[0]) * (box1[3] - box1[1])
    box2_area = (box2[2] - box2[0]) * (box2[3] - box2[1])

    # Union area
    union_area = box1_area + box2_area - intersection_area

    # IoU
    # If union_area > 0, calculate IoU. Otherwise, return 0.0.
    iou = tf.where(union_area > 0, intersection_area / union_area, 0.0)

    return iou

iou = compute_iou(box1, box2)
print(f"IoU: {iou.numpy():.4f}")
```

```
IoU: 0.2195
```

By using Union Area $=$ Area$_1$ $+$ Area$_2$ $-$ Intersection Area, we are removing the duplicate counting of the intersection.

IoU

TensorFlow already has a built-in function to calculate Intersection over Union (IoU). We do not need to write the entire calculation manually.

```python
import tensorflow as tf
import keras


# Define boxes with the correct shape (1, 4)
box1 = tf.constant([[0.20, 0.20, 0.70, 0.70]], dtype=tf.float32)
box2 = tf.constant([[0.40, 0.40, 0.90, 0.90]], dtype=tf.float32)

# Calculate IoU
iou = keras.utils.bounding_boxes.compute_iou(
    box1,
    box2,
    bounding_box_format="rel_yxyx",
    image_shape=(720, 1280, 3)
)

print(iou.numpy())
```

```
[[0.2195122]]
```

Bounding Box Formats in Keras

- **xyxy:** $[x_{\text{min}}, y_{\text{min}}, x_{\text{max}}, y_{\text{max}}]$ Top-left and bottom-right corners.
- **yxyx:** $[y_{\text{min}}, x_{\text{min}}, y_{\text{max}}, x_{\text{max}}]$ Same corners, Y coordinates first.
- **xywh:** $[x_{\text{min}}, y_{\text{min}}, w, h]$ Top-left corner, width, and height.
- **center_xywh:** $[x_c, y_c, w, h]$ Center coordinates, width, and height.
- **Relative formats (rel_):** Coordinates normalized between 0 and 1.
  - `rel_xyxy`, `rel_yxyx`
  - `rel_xywh`, `rel_center_xywh`
- **Example:** Box $(20, 30, 70, 80)$ in a 100×100 image:

$$\begin{aligned} \text{xyxy} &= [20, 30, 70, 80] \\ \text{xywh} &= [20, 30, 50, 50] \\ \text{center_xywh} &= [45, 55, 50, 50] \\ \text{rel_xyxy} &= [0.2, 0.3, 0.7, 0.8] \end{aligned}$$

Current Subsection

1. Object Detection
   - Motivation
   - Bounding Boxes
   - Intersection over Union
   - Non-Maximum Suppression
   - Multiple Classes
   - Detector Architectures

Why Do We Need Non-Maximum Suppression?

Object detectors may predict several overlapping bounding boxes for the same object.

For example, one car may produce:

$$\text{Box}_1, \quad \text{Box}_2, \quad \text{Box}_3$$

with different confidence scores.

Problem

We do not want to keep multiple detections for the same object.

Solution

Non-Maximum Suppression (NMS) keeps the best box and removes strongly overlapping duplicates.

Code

```python
import tensorflow as tf

# TensorFlow box format:
# [ymin, xmin, ymax, xmax]

boxes = tf.constant([
    [0.10, 0.10, 0.50, 0.50],
    [0.12, 0.12, 0.52, 0.52],
    [0.60, 0.60, 0.90, 0.90]
], dtype=tf.float32)

scores = tf.constant([
    0.95,
    0.85,
    0.80
], dtype=tf.float32)

selected_indices = tf.image.non_max_suppression(
    boxes=boxes,
    scores=scores,
    max_output_size=10, # Keep at most 10 boxes after NMS
    iou_threshold=0.5 # If a lower-scoring box overlaps too much with a higher-scoring box, specifically IoU greater than 0.5, suppress it.
)

print("Selected indices:", selected_indices.numpy())
```

```
Selected indices: [0 2]
```

If more than 10 boxes still survive, then the 10 highest-scoring surviving boxes are kept.

How Does Non-Maximum Suppression Work?

NMS repeatedly performs the following steps:

1. Select the box with the highest confidence score.
2. Keep that box.
3. Compute IoU with the remaining boxes.
4. Remove boxes whose IoU is above a threshold.
5. Repeat until no boxes remain.

Key Idea

NMS keeps the strongest detection and suppresses nearby duplicates.

Code

```
selected_boxes = tf.gather(boxes, selected_indices)
selected_scores = tf.gather(scores, selected_indices)

print("Selected boxes:")
print(selected_boxes.numpy())

print("\nSelected scores:")
print(selected_scores.numpy())
```

```
Selected boxes:
[[0.1 0.1 0.5 0.5]
 [0.6 0.6 0.9 0.9]]

Selected scores:
[0.95 0.8 ]
```

NMS removed the box with confidence 0.85 because its IoU with the higher-confidence box (0.95) exceeded the 0.5 threshold, while the box with confidence 0.80 was kept because it did not overlap significantly.

NMS IoU Threshold

The IoU threshold controls how aggressively overlapping boxes are removed.

- Lower threshold: removes more overlapping boxes
- Higher threshold: keeps more overlapping boxes

For example:

$$\texttt{iou_threshold} = 0.5$$

means that boxes overlapping more than 50% may be suppressed.

Important

The best threshold depends on the detector and the application.

Confidence Threshold

Object detectors often produce many candidate boxes.

Each box has a confidence score:

$$0 \leq \text{score} \leq 1$$

A **confidence threshold** removes predictions with low confidence.

$$\text{keep box if score} \geq T$$

Example

If $T = 0.5$, predictions with confidence below 0.5 are discarded.

Code

```
confidence_threshold = 0.5

keep = scores >= confidence_threshold

filtered_boxes = tf.boolean_mask(boxes, keep)
filtered_scores = tf.boolean_mask(scores, keep)

print("Filtered boxes:")
print(filtered_boxes.numpy())

print("\nFiltered scores:")
print(filtered_scores.numpy())
```

```
Filtered boxes:
[[0.1  0.1  0.5  0.5 ]
 [0.12 0.12 0.52 0.52]
 [0.6  0.6  0.9  0.9 ]]

Filtered scores:
[0.95 0.85 0.8 ]
```

Confidence Threshold + NMS

A common post-processing pipeline is:

$$\boxed{\text{Candidate Boxes} \rightarrow \text{Confidence Threshold} \rightarrow \text{NMS} \rightarrow \text{Final Detections}}$$

- Confidence threshold removes weak predictions.
- NMS removes duplicate overlapping predictions.

Key Idea

Detection models usually need post-processing before producing final results.

Code

```
selected_indices = tf.image.non_max_suppression(
    boxes=filtered_boxes,
    scores=filtered_scores,
    max_output_size=10,
    iou_threshold=0.5
)

final_boxes = tf.gather(filtered_boxes, selected_indices)
final_scores = tf.gather(filtered_scores, selected_indices)

print("Final boxes:")
print(final_boxes.numpy())

print("\nFinal scores:")
print(final_scores.numpy())
```

```
Final boxes:
[[0.1 0.1 0.5 0.5]
 [0.6 0.6 0.9 0.9]]

Final scores:
[0.95 0.8 ]
```

Current Subsection

1. Object Detection
   - Motivation
   - Bounding Boxes
   - Intersection over Union
   - Non-Maximum Suppression
   - Multiple Classes
   - Detector Architectures

Object Detection with Multiple Classes

Real object detectors usually predict many object classes.

For each candidate detection, the model may output:

bounding box

class scores

confidence score

Note: The class score represents the model's confidence that the detected object belongs to that specific class.

For example: car : 0.91 person : 0.04 truck : 0.03

Prediction

The class with the highest score becomes the predicted label.

Class score: How likely the detected object belongs to a particular class, such as car, person, or truck.
Objectness confidence: How likely the bounding box contains an object at all, regardless of its class.

Code

```python
import tensorflow as tf

class_names = ["car", "person", "truck"]

class_scores = tf.constant([
    [0.91, 0.04, 0.03],
    [0.10, 0.85, 0.05],
    [0.25, 0.15, 0.60]
], dtype=tf.float32)

predicted_class_ids = tf.argmax(class_scores, axis=1)

print(predicted_class_ids.numpy())

for i, class_id in enumerate(predicted_class_ids.numpy()):
    print(f"Detection {i}: {class_names[class_id]}")
```

```
[0 1 2]
Detection 0: car
Detection 1: person
Detection 2: truck
```

Why NMS Is Usually Applied Per Class

Suppose a car box overlaps strongly with a pedestrian box.

They may describe different objects, so one should not automatically suppress the other.

Therefore, object detection systems often apply NMS separately for each class.

Key Idea

Suppress duplicate **car** boxes against other car boxes, and duplicate **person** boxes against other person boxes.

TensorFlow Combined Non-Maximum Suppression

TensorFlow provides:

```
tf.image.combined_non_max_suppression()
```

This function can handle:

- many candidate boxes,
- multiple classes,
- confidence scores,
- per-class NMS,
- final detection limits.

Why Useful?

This is closer to the post-processing used in real object detectors.

Code

```
import tensorflow as tf

boxes = tf.constant([
    [
        [[0.10, 0.10, 0.50, 0.50]],
        [[0.12, 0.12, 0.52, 0.52]],
        [[0.60, 0.60, 0.90, 0.90]]
    ]
], dtype=tf.float32)


scores = tf.constant([
    [
        [0.95, 0.05], # Each candidate box has one score for each class.
        [0.85, 0.10],
        [0.10, 0.90]
    ]
], dtype=tf.float32)
```

Code—Continued

```
nms_output = tf.image.combined_non_max_suppression(
    boxes=boxes,
    scores=scores,
    max_output_size_per_class=10,
    max_total_size=10,
    iou_threshold=0.5,
    score_threshold=0.5
)

print("Boxes:")
print(nms_output.nmsed_boxes.numpy())

print("\nScores:")
print(nms_output.nmsed_scores.numpy())

print("\nClasses:")
print(nms_output.nmsed_classes.numpy())

print("\nValid detections:")
print(nms_output.valid_detections.numpy())
```

max_output_size_per_class=10 means NMS can keep up to 10 detections for each individual class, such as 10 cars and 10 persons, while max_total_size=10 means that after combining detections from all classes, TensorFlow will return at most 10 detections in total.

Code—Result

```
Boxes:
[[[0.1 0.1 0.5 0.5]
  [0.6 0.6 0.9 0.9]
  [0.  0.  0.  0. ]
  [0.  0.  0.  0. ]
  [0.  0.  0.  0. ]
  [0.  0.  0.  0. ]
  [0.  0.  0.  0. ]
  [0.  0.  0.  0. ]
  [0.  0.  0.  0. ]
  [0.  0.  0.  0. ]]]

Scores:
[[0.95 0.9  0.   0.   0.   0.   0.   0.   0.   0.  ]]

Classes:
[[0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]]

Valid detections:
[2]
```

Summary: What Does NMS Try to Keep?

Non-Maximum Suppression's goal is to remove multiple overlapping boxes that likely represent the same object.

Main Idea

One final box per detected object, ideally

For example, if an image contains:

Dog Cat Car

the detector may initially create several candidate boxes for each object.
NMS keeps the strongest box for each object and suppresses weaker, highly overlapping boxes.

In many object detectors, NMS is also applied **per class**, so dog boxes are compared with dog boxes, cat boxes with cat boxes, and so on.
(NMS does not truly know the object labels itself; it relies on the object detector's predicted classes and confidence scores, together with IoU, to identify likely duplicate boxes.)

Object Detection Pipeline So Far

We now understand the basic output-processing pipeline:

$$\boxed{\text{Model} \rightarrow \text{Candidate Boxes} \rightarrow \text{Class Scores} \rightarrow \text{Confidence Filtering} \rightarrow \text{NMS} \rightarrow \text{Final Detections}}$$

Final detections contain:

$$\boxed{\text{class} + \text{confidence} + \text{bounding box}}$$

Current Subsection

1. Object Detection
   - Motivation
   - Bounding Boxes
   - Intersection over Union
   - Non-Maximum Suppression
   - Multiple Classes
   - Detector Architectures

Two Main Families of Object Detectors

Modern object detectors are often grouped into two categories:

Two-Stage Detectors

First find candidate object regions, then classify them.
Examples:

- R-CNN
- Fast R-CNN
- Faster R-CNN

One-Stage Detectors

Predict boxes and classes directly in one pass.
Examples:

- SSD
- YOLO
- RetinaNet

One-Stage vs. Two-Stage Detection

Aspect	One-Stage	Two-Stage
Pipeline	Direct prediction	Proposals + classification
Speed	Usually faster	Usually slower
Complexity	Simpler inference	More complex
Real-time use	Very common	Less common
Examples	YOLO, SSD	Faster R-CNN

Practical Idea

One-stage detectors are often preferred for real-time applications, while two-stage detectors are useful when high localization quality is important.

Congratulations!

Now you know the basics of object detection!

